

C10 — UX and Product Quality

1. What interviewers test

Product interviews rarely ask for a design critique in the vocabulary of design, and they reward it when it appears. A candidate who says "this screen has no error state, and the label says 'Error 4021', which tells the user nothing about what to do" is easier to trust than one who says "it feels clunky." The interviewer is not looking for a designer. They are looking for a PM who can see why an interface fails, name the problem precisely, weigh the quality tradeoffs a team actually faces, notice when a design manipulates people, and work well with the people who design.

Where it shows up. Inferred from the round structures candidates commonly report: critique, "improve this screen" and design prompts at Google, Meta, Microsoft, Amazon, Apple and others, where the interviewer listens for UX reasoning inside a product-sense answer, and behavioral questions about working with designers and handling disagreements [unverified: candidate-reported]. This chapter supplies the vocabulary that C11 (critique and teardown) uses.

What is scored. Whether your reasoning about interfaces is specific and user-centered Inferred: do you walk the flow as a user, name the principle at stake, consider the states other than the happy path, think of people who use the product differently from you, propose a way to test, and know where to spend craft and where not to.

Six observable behaviors:

1. You walk a flow from the user's task, not from the screen.
2. You name the problem with a principle (a heuristic, an affordance, a cognitive-load issue) and say what it costs the user.
3. You check the states beyond the ideal: empty, loading, error, offline, permission denied.
4. You consider accessibility and people who differ from you (ability, language, device, situation).
5. You decide how much quality a feature deserves and say why.
6. You spot manipulative patterns, and you say what you would do instead.

The failure this prevents. Six patterns weaken UX answers: "it feels clunky" with no principle, a critique of visual taste only, designing only the ideal path, ignoring accessibility, polishing everything equally (or nothing), and optimizing a number by making the interface pushier.

2. The concept from first principles

C10.1 The PM's role in UX

A PM is not the designer, and is responsible for the outcome the design serves. The useful split: the PM owns the *problem* (who, what they are trying to do, what success looks like, the constraints), the designer owns the craft of the *solution* (how it looks, flows and feels), and the engineer owns what can be built well. The PM's contribution to UX is a clear problem statement ([[problem-statement]], C06), the user evidence, the success criteria, the tradeoffs and decisions, and honest feedback on whether a design solves the problem.

What a PM should not do. Dictate the layout from taste ("make the button green and bigger"). A designer who is told the problem and the criteria ("a first-time user must find the checkout in one step, and we cannot add a tab") can use their craft; one who is handed a solution can only execute it.

When not to rely on UX principles alone. Heuristics and guidelines find usability problems in a design; they do not tell you whether people want the product or whether you chose the right problem (C06 and C07). A flawless interface for an unwanted feature is still a failure.

Counterexample. A PM reviewing a signup form says, "I don't like the layout. Move the password field to the top and make the button blue." The designer complies. Completion does not change. A PM who had said "Of the people who start signup, 40% leave at the password step; here are three recordings where people stall on the rules; the goal is 70% completion" would have given the designer something to solve, and the designer might have shown the rules while typing instead of after an error.

Hidden risk. Mistaking your own preference for a user finding. Your taste is one data point; usability testing (C10.7) is how you replace it with evidence.

C10.2 Usability heuristics and design principles

A **usability heuristic** is a rule of thumb for judging whether an interface is easy to use, applied by an evaluator without users. The best known set is ten heuristics from Jakob Nielsen (FW-C10-01), which this chapter defines once and C11 reuses. In my words:

#	Heuristic	The question
1	Visibility of system status	Does the user always know what is happening (saving, loading, sent)?
2	Match with the real world	Does it use the user's words and concepts, not internal jargon?
3	User control and freedom	Can the user undo, go back and escape?
4	Consistency and standards	Does the same thing look and act the same, and match platform conventions?
5	Error prevention	Does the design stop mistakes before they happen?

#	Heuristic	The question
6	Recognition over recall	Is what the user needs visible, not memorized?
7	Flexibility and efficiency	Are there shortcuts for experts without burdening novices?
8	Minimal design	Does every element earn its place?
9	Help with errors	Do errors say what went wrong and how to fix it, in plain words?
10	Help and documentation	Is help findable when needed?

Cognitive load. **Cognitive load** is the mental effort a task demands of working memory; interfaces that add unneeded effort slow people and cause errors. Many heuristics reduce it: recognition over recall, minimal design, consistency.

Norman's concepts. Don Norman's vocabulary (C07's further reading) complements the list: an *affordance* is what an object lets you do; a *signifier* is the cue that shows it (a button that looks pressable); *feedback* confirms an action took place; *constraints* limit wrong actions; *mapping* makes controls match their effects; a *conceptual model* is the picture of how it works that the user forms.

Why it works. Named principles let you say precisely what is wrong and persuade a team without relying on taste. **When not to use them.** A heuristic review cannot replace watching users, and heuristics sometimes conflict (minimal design against visibility of status); say which you favor and why.

C10.3 Platform design guidelines as public references

Operating-system makers publish design guidelines: Apple's Human Interface Guidelines, Google's Material Design, Microsoft's Fluent design system `Public [V: design-system lists naming Apple's Human Interface Guidelines, Google's Material Design and Microsoft's Fluent, with their addresses]`. They describe conventions for navigation, controls, spacing, motion and accessibility on each platform. Use them as references for consistency: people learn platform conventions, and an app that follows them is easier to learn. Say "I'd check this against the platform's guidelines" and name the principle (standards, consistency), not specific rules you cannot recall. Deviate deliberately, for a reason (a brand-defining interaction, a task the standard control serves poorly) and test the deviation.

C10.4 Information architecture and interaction patterns

Information architecture is the way content and functions are organized, grouped and labeled so people can find them. It decides whether a person finds settings under "Account" or "Preferences," and whether a product with many features feels simple. Good IA starts with the user's words and tasks. Methods (awareness depth): *card sorting* (users group items and name the groups) and *tree testing* (users try to find items in a proposed structure).

Common patterns. Navigation (a bottom tab bar for a few top-level areas on phones, a menu for many, search for a large catalog); forms (few fields, clear labels, inline validation, the right keyboard for the input); onboarding (teach by doing, show value early, avoid long

tours); lists and filters; *progressive disclosure* (show the common first and reveal the rare on request); confirmation and undo (prefer undo to "are you sure?" when the action is reversible); search with good empty results.

Why it works. Patterns encode what people already know, which lowers cognitive load.

When not to follow them. When the task is unusual, and then test.

C10.5 States and edge cases: empty, error, loading

Most design attention goes to the ideal state, when the data is there and everything works. Real users spend much of their time elsewhere. FW-C10-02 lists the states to design.

State	The question	Typical failure
Ideal	Does it work when all is well?	None; this is the one everyone designs
Empty	What does a new user see with no data?	A blank screen with no next step
Loading	What shows while waiting?	A frozen screen; a spinner with no end
Partial	What if some data failed to load?	The whole page fails
Error	What does a failure say and offer?	"Something went wrong", a code, no action
Offline or slow	Does it work on a bad connection?	Silent failure
Permission denied	What if the user said no to location or notifications?	The feature dies without explanation
Success	Does the user know it worked?	No confirmation, so users repeat the action

An empty state is a chance to teach ("No projects yet. Create your first one"); an error state is a chance to recover ("We couldn't save. Check your connection; we'll try again, and your work is kept"). PMs who ask "what happens when this is empty, and when it fails?" in every review prevent a large share of poor experiences.

C10.6 Accessibility and inclusive design

Accessibility is the degree to which a product can be used by people with the widest range of abilities, including through assistive technology (screen readers, switch controls, captions, voice control). **Inclusive design** is the broader practice of designing with people who differ from the typical user, and it uses the curb-cut idea from C07: a feature built for a constraint often helps everyone (captions help in a noisy room).

A pass you can run (FW-C10-03). The W3C's accessibility guidelines organize requirements into four principles: content must be *perceivable*, interfaces *operable*, content and controls *understandable*, and the whole *robust* enough for assistive technology [Public \[V: W3C: WCAG 2.2 is the current Recommendation\]](#). Concretely:

- **Contrast.** Text needs enough contrast with its background. The guideline's usual minimum is a ratio of 4.5 to 1 for normal text and 3 to 1 for large text. Grey #767676 on

white measures 4.54:1 and passes; #777777 measures 4.48:1 and fails by a hair; #959595 measures 3.0:1, which passes only for large text. A designer can check this in seconds.

- *Do not rely on color alone* (an error shown only in red excludes people with color-vision differences).
- *Targets and touch*. Controls large enough to hit, with space between them.
- *Labels for assistive technology*. Every control has a name a screen reader can speak; images have text alternatives.
- *Text size and motion*. The interface survives larger text and respects a reduced-motion setting.
- *Captions and transcripts* for audio and video.
- *Keyboard and voice operation* for everything.

Situations, not only disabilities. A person with one arm in a cast, a parent holding a baby, a user in bright sun and a user on a slow connection all meet the same barriers as a person with a permanent disability. Designing for the permanent case serves the temporary and situational ones (the inclusive-design framing published by Microsoft [Public](#) [V: Microsoft Inclusive Design toolkit manual]).

Regional Lens: India. The contexts that matter most in many Indian markets are low bandwidth, low literacy and shared devices Inferred [unverified: author's synthesis; no source checked]. Practical consequences: icons and voice in the main path, short text in several languages, light pages that work on slow connections, and profiles or privacy on a phone shared by a family (C09 develops these). They are accessibility needs for a large part of the population and not edge cases.

Why it works. A short, concrete pass catches the common barriers, and an accessible design is usually a clearer design for everyone. **When not to treat it as a checklist only.** Automated checks find a minority of problems; testing with people who use assistive technology finds the rest.

C10.7 Usability testing for PMs

A usability test shows people trying to do real tasks with the product and lets you watch where they struggle. FW-C10-04 is a lightweight protocol.

The method. Pick the three to five tasks that matter most. Recruit five or six people who match the target segment. Ask each to do the tasks while thinking aloud; you watch, you do not help, and you note where they hesitate, err or give up. Rate each problem by severity (how bad, how many people, how likely to recur). Fix the severe ones, then test again.

Why five. Problems are found with diminishing returns. If each participant finds about 31% of the problems on average (the figure reported by Nielsen and Landauer [V: Nielsen and Landauer (1993): one user finds about 31 percent of problems]), five participants find $1 - 0.69^5 \approx 84\%$ of them, and ten find about 98%. So small, repeated rounds beat one large study, and the first five people find the worst problems.

What it cannot do. It does not tell you how many people have a problem (use analytics, C05) or whether people want the product (C06). Observe behavior, not opinions: what people do differs from what they say ([say-do-gap], C06). Do not lead ("click the red button"); ask "what would you do next?"

Why it works. Watching five people fail at a task is more persuasive than any argument.

When not to test. A trivial copy change, or a decision already settled by data; test when a design is new, complex or high-stakes.

C10.8 Quality tradeoffs: speed, craft, delight

Teams cannot polish everything. FW-C10-05 is a dial for where to invest. Quality has three layers: *works* (correct and reliable), *usable* (clear and efficient), and *delightful* (polish that makes people like it). Spend the most where the cost of failure is highest or the moment matters most: the core task, moments of trust (payment, privacy, account recovery), first-run experience, and accessibility. Accept rougher quality where the audience is small or internal, where you are testing an idea (an [mvp], C15) or where speed matters more than polish.

Polish debt. Rough edges shipped for speed accumulate and eventually slow users and teams down; schedule time to clear them, as with technical debt.

Why it works. A stated quality bar turns "make it nicer" into a decision with a reason.

When not to compromise. On accessibility basics, safety, and honest information; these are not polish.

C10.9 Behavior and motivation: habits, nudges, choice architecture

Interfaces shape behavior. A **nudge** is a design choice that steers behavior toward an option without forbidding others or changing incentives sharply: a default, the order of options, a reminder, a message about what others do. *Choice architecture* is the design of how options are presented. Defaults have a large effect on what people end up with (C07's defaults model), so the default is often the decision.

Habit loops. Many products build habits through a cue, an action and a reward (the habit loop of cue, routine and reward, popularized by Charles Duhigg's *The Power of Habit*, 2012, and extended in Nir Eyal's *Hooked*, 2014 [V: Duhigg, *The Power of Habit*, Random House, 2012: cue, routine, reward; Eyal, *Hooked*, 2014: trigger, action, variable reward, investment]). Used well, a loop helps people do something they want to do (a medication reminder). Used badly, it drives compulsive use.

Ethical limits (FW-C10-06). A nudge is acceptable if it is *transparent* (the user could see what is being done), *reversible* (easy to undo), *in the user's interest* (they would endorse it if they understood it), and *symmetric* (declining is as easy as accepting). A pre-selected organ-donor choice with an easy opt-out is a nudge; a pre-checked box that signs people up for recurring charges and hides the way out is not.

C10.10 Dark patterns and design ethics

A **dark pattern** is a user-interface design that tricks or pressures people into actions they did not intend, to the benefit of the business. Common forms:

Pattern	What it does
Hard to cancel ("roach motel")	Easy to enter, hard to leave
Forced continuity	A free trial that silently becomes a paid plan
Hidden costs	Fees revealed only at the last step
Pre-checked boxes	Opt-in by default for things people would not choose
Confirmshaming	Making the decline option sound foolish ("No, I don't like saving money")
Misdirection	Making the profitable choice visually dominant and the other hard to see
Nagging	Repeated prompts until the user gives in

Why teams build them. A metric rewards them: conversion or retention rises. This is [[goodharts-law]] (C05): when the measure becomes the target, it can be raised by hurting the user. The cost shows up later, as complaints, chargebacks, lost trust and, increasingly, enforcement: regulators in several jurisdictions treat some of these designs as unlawful [V: India's Central Consumer Protection Authority guidelines of 30 November 2023 list 13 dark patterns as unfair trade practices; the EU Digital Services Act bars deceptive interface design by online platforms; the US case is in section 10], and section 10 describes a major US case.

What a PM does. Name the pattern, show the cost beyond the metric (refunds, support, churn later, risk), propose an honest alternative that serves the business goal (a clear value reminder, a pause option, an easy exit), and measure net outcomes (revenue after refunds, retention of people who chose to stay), not only the number the pattern lifts. The responsible-design check in C07 (FW-C07-06) applies.

C10.11 Working with designers

A good working relationship has these habits. Share the problem and evidence early and the constraints honestly. Critique designs against the problem ("does this help a first-time user find checkout?"), not against taste. Ask for options and the reasoning behind them. Attend research sessions.

Bring engineers into design reviews early so cost and feasibility surface before a design is polished. Respect the design system (a shared set of components and rules) as a way to stay consistent and move faster, and ask for an exception only with a reason. Agree who decides:

the PM decides what to build and the success criteria; the designer leads on how it should look and work; disagreements go to evidence (a quick test) before rank. C32 develops influence without authority.

3. Frameworks

Six frameworks, set out below. The responsible-design check (FW-C07-06), the surface lens (FW-C07-07), the HEART framework (FW-C05-08) and the interview guide (FW-C06-04) are reused, not repeated.

FW-C10-01 · The heuristic review · Established

- **Origin:** Jakob Nielsen's ten usability heuristics, developed with Rolf Molich in 1990 and refined in 1994, published by Nielsen Norman Group [V: Nielsen Norman Group article; developed with Rolf Molich 1990, refined 1994].
- **Inputs:** a flow or screen and a user task.
- **Outputs:** a list of problems, each tied to a heuristic, with a severity and a proposed fix.
- **Assumptions:** an evaluator can walk the task as a user would; two or three evaluators find more than one.
- **Tradeoffs:** quick and cheap, and no substitute for watching users; evaluators share blind spots.
- **Failure modes:** reviewing the screen and not the task; naming no severity; listing taste issues as problems.
- **Worked example:** the settings-screen review in Worked example 1.

Steps: (1) state the user and task; (2) walk the task step by step; (3) at each step test the ten heuristics (C10.2); (4) log each problem with the heuristic, the cost to the user and a severity (blocks the task, slows it, annoys); (5) propose fixes in order of severity.

FW-C10-02 · The state map · Original

- **Inputs:** a screen or feature with data and actions.
- **Outputs:** a table of the eight states with what the user sees and can do in each, and gaps marked.
- **Assumptions:** the states can be enumerated from how the feature gets its data.
- **Tradeoffs:** designing every state costs time; skip those that cannot occur.
- **Failure modes:** only the ideal state; an error with no recovery; an empty state with no next step.
- **Worked example:** the search results page in Worked example 2.

States: **ideal, empty, loading, partial, error, offline or slow, permission denied, success.**

FW-C10-03 · The accessibility pass · Established

- **Origin:** the perceivable, operable, understandable and robust principles of the W3C Web Content Accessibility Guidelines [V: W3C: WCAG 2.2 is the current Recommendation].
- **Inputs:** a design or build.
- **Outputs:** a list of barriers found against seven checks, each with a fix, and a note on what needs testing with assistive technology.
- **Assumptions:** tools and people who use assistive technology are available for the final check.
- **Tradeoffs:** a quick pass finds common problems and misses subtle ones.
- **Failure modes:** treating accessibility as a late checklist; relying on automated tools alone; designing for color alone.
- **Worked example:** the contrast and labeling checks in EX-C10-03.

Checks: **contrast, not color alone, target size, labels for assistive technology, text size and motion, captions and transcripts, keyboard and voice operation.**

FW-C10-04 · The five-user usability test · Original

- **Inputs:** a prototype or product, three to five tasks, five or six people from the target segment.
- **Outputs:** a ranked list of problems with severity, a task success rate, and the fixes to make before the next round.
- **Assumptions:** participants resemble real users and think aloud.
- **Tradeoffs:** small and fast; it finds problems and does not size them.
- **Failure modes:** leading participants; helping them; testing friendly users; treating five people as a statistic.
- **Worked example:** the test read-out in Worked example 3.

Protocol: **tasks → recruit → think aloud, do not help → log hesitations, errors, give-ups → rate severity → fix the severe → retest.**

FW-C10-05 · The quality dial · Original

- **Inputs:** a feature, its audience, and the cost of failure.
- **Outputs:** a quality bar for each of works, usable and delightful, with a reason, and a list of accepted rough edges.
- **Assumptions:** the team can estimate the cost of failure and the audience.
- **Tradeoffs:** a lower bar ships faster and creates polish debt.
- **Failure modes:** polishing everything equally; cutting accessibility or safety; no record of accepted rough edges.
- **Worked example:** IQ-C10-03.

Layer	High bar when	Lower bar when
Works	Always	Never
Usable	Core task, payments, privacy, first run	Small internal audience
Delightful	A brand-defining moment, a differentiator	An experiment, an internal tool

FW-C10-06 · The ethical nudge test · Original

- **Inputs:** a design that steers behavior (a default, a prompt, a flow).
- **Outputs:** answers to four questions, a verdict (acceptable, change, remove), and an honest alternative.
- **Assumptions:** you can describe the design as the user would experience it.
- **Tradeoffs:** a stricter test costs some short-term conversion.
- **Failure modes:** testing only the metric; judging the design by intent and not by effect; no alternative offered.
- **Worked example:** the cancellation flow in Worked example 4, and IQ-C10-05.

Question	Test
Transparent	Could the user see what is being done and why?
Reversible	Is it as easy to undo as to do?
In the user's interest	Would they endorse it if they understood it?
Symmetric	Is declining as easy as accepting?

4. Answer architecture

The UX-reasoning answer, for a 20-minute segment

Step	Minutes	What to produce
User and task	2	The person, what they are trying to do, the context
Walk the flow	6	The heuristic review: problems tied to a principle, with severity
States and edge cases	3	The state map: empty, loading, error, offline, permission
Accessibility and inclusion	3	The pass: contrast, labels, targets, situations of use
Prioritize and propose	4	The one or two fixes that matter, with the reason, and the quality bar
Test and measure	2	A five-user test and a metric with a guardrail

Pacing rules. If the prompt gives a screen, spend the time on the walk and the states. If you run behind, keep the severity ranking and the test. Treat the interviewer's questions as steering.

How to say it. Name the principle with the cost: "The error says 'Error 4021', which violates 'help with errors': the user cannot tell what to do, so I'd expect them to abandon." Then the fix.

How to close. Two sentences: "The biggest problem is [problem], which costs the user [cost]; I'd fix it by [fix], test it with five users, and measure [metric] with [guardrail]. I'd treat [accessibility or ethical point] as a requirement, not a nicety."

Type: critique a screen or flow

Use FW-C10-01 and FW-C10-02. C11 turns this into a full critique method.

Type: quality tradeoff

Use the quality dial (FW-C10-05): where the cost of failure is high, spend; where it is low, accept rough edges and record them.

Type: manipulation or ethics

Use the ethical nudge test (FW-C10-06), name the pattern and the costs beyond the metric, and offer an honest alternative.

Type: working with a designer

Describe the problem you gave, the evidence, how you critiqued against the problem and how a disagreement was settled (a quick test).

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"Why is that a problem?"	You named a feeling	Name the principle and what it costs the user
"What if the list is empty?"	You skipped states	Walk the state map
"Who might not be able to use it?"	Accessibility	Give two barriers and fixes
"We don't have time to polish."	Quality tradeoff	State the bar by layer and what you accept
"Conversion would go up with that."	A metric argument for a dark pattern	Name the pattern, the cost beyond the metric, an alternative
"How do you know users struggle?"	Opinion versus evidence	Propose a five-user test and an analytics check

5. Worked examples

All four examples are **Illustrative**: the products are fictional and every figure is an invented assumption. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — A heuristic review of a notification-settings screen (Illustrative)

The screen (described). A phone app's notification settings (invented): one long list of 22 toggles with short labels such as "Enable?" and "Mentions"; an "Off" state shown in red text; touch targets about 24 pixels tall; a "Reset all" link at the top that acts at once; no search; and toggle styling that differs from the phone's standard switch.

The user and task. A person who is being interrupted too often wants to turn off most alerts but keep messages from family. Task: find and change two settings in under a minute.

The review (FW-C10-01).

Problem	Heuristic	Cost to the user	Severity
"Enable?" has no object, so it is unclear what it controls	2 Match with the real world; 1 Visibility	Guessing; wrong changes	Slows the task
22 toggles in one list with no grouping or search	6 Recognition over recall; 8 Minimal design	Scanning effort (cognitive load); abandonment	Slows the task
"Reset all" acts at once with no confirmation or undo	3 User control and freedom; 5 Error prevention	A destructive mistake with no recovery	Blocks (data loss)
Off is shown only as red text	Accessibility: not relying on color alone	People with color-vision differences cannot tell state	Blocks for some
Targets about 24 pixels tall	Accessibility: target size; Fitts's law	Mis-taps, especially on the move	Slows the task
Custom toggle style	4 Consistency and standards	The control looks unfamiliar, so people hesitate	Annoys

What I'd fix first. The destructive "Reset all" (add undo, or confirm), then group the toggles by purpose with a search, then label each toggle by what it controls, then fix target size and the on/off cue. I'd check the result against the platform's published guidelines for switches and lists [Public \[V: Apple Human Interface Guidelines and Material Design\]](#), which is a consistency question (heuristic 4).

How I'd confirm. A five-user test on the task "turn off everything except messages from family" (FW-C10-04), watching for hesitation at the labels and the mis-taps.

What this shows. Each problem was tied to a principle and a cost, ranked by severity, and the fix order followed the severity, not the order in which the issues were noticed.

Worked example 2 — A state map for a search results page (Illustrative)

The feature. A recipe app's search results page (invented). The ideal state is a list of recipes with photos. What about the other states (FW-C10-02)?

State	What the user sees today	What it should be
Ideal	A list of recipes	Fine
Empty	A blank page for "chickpea tagine"	"No recipes for 'chickpea tagine'. Try 'chickpea stew', or browse popular recipes." with a clear next step
Loading	A frozen page for several seconds	A placeholder layout at once; after about three seconds, a message that it is still loading
Partial	The whole page fails if one photo fails	Show the text and a placeholder for the missing photo
Error	"Something went wrong"	"We can't reach the server. Your saved recipes are available offline. Retry."
Offline or slow	A silent failure	Show cached results and say they may be out of date
Permission denied	"Near me" does nothing after location was denied	"Turn on location to see nearby stores, or enter a postcode"
Success	Tapping "Save" changes nothing visibly	A brief confirmation with Undo

Priorities. The empty and error states matter most because they are the ones users meet when something is already going wrong, and a dead end there loses the user. The offline state matters in markets where connections are unreliable (C09). The success state prevents repeated taps and duplicate actions.

What I'd ask in every review. "What does this show when it is empty, and when it fails?" It takes ten seconds and surfaces most of the table.

What this shows. Designing only the ideal state leaves most of the user's real experience unspecified.

Worked example 3 — Reading a five-user usability test (Illustrative)

The test. Five people from the target segment try four tasks on a checkout prototype (invented). Results:

Task	Succeeded	What was seen
Find a product	5 of 5	Smooth
Apply a coupon code	2 of 5	Three people did not find the field; it sat behind a grey "Have a code?" link with contrast of 2.9 to 1
Check out as a guest	4 of 5	One person assumed an account was required
Change the delivery address	3 of 5	Two people edited the billing address instead; the labels were similar

Read it (FW-C10-04). Rank by severity. The coupon field is the biggest problem: three of five failed, the cause is visible (a hidden, low-contrast link), and it blocks the task. The address task is next: two wrong edits could send a parcel to the wrong place. The guest-checkout problem is a labeling fix.

What the test can and cannot say. Five people reveal the severe problems. If each participant finds about 31% of the problems (the commonly reported figure [V: Nielsen and Landauer (1993): one user finds about 31 percent of problems]), five find about 84% of them, so most of what is wrong has been seen. It does not size the problem: three failures out of five gives a rate of 60%, but the 95% interval for that rate is roughly 23% to 88%, which says only that failure is common. To learn how many real customers fail, use analytics on the live product (the share who open the coupon link and complete). Do not announce "60% of users fail."

Fixes. Make the coupon field visible with adequate contrast (4.5:1 or better for normal text, C10.6), rename the two addresses clearly ("Delivery address", "Billing address"), and say "Continue as guest" first. Retest with five new people, since the first round used up the participants who now know the answers.

What this shows. A small test is excellent at finding problems and poor at counting them, and the result should be stated with that limit.

Worked example 4 — A cancellation flow that makes leaving hard (Illustrative)

The situation. A subscription app's cancellation takes six steps across four screens, with offers and warnings at each (invented). Each month, 5,000 subscribers try to cancel. The current flow lets 62% finish and 38% give up or fail. A designer proposes a two-step flow that would let 95% finish.

What the numbers say. Today 3,100 of 5,000 cancel and 1,900 stay. In the new flow 4,750 cancel and 250 stay. So 1,650 more people would cancel each month. At \$12 a month, that is \$19,800 of monthly revenue that would stop. The growth lead argues to keep the current flow.

The honest accounting. The 1,650 are people who tried to leave and could not. They do not stay because they value the product; they stay because they were stopped. Some consequences (invented rates):

- 15% of them dispute the charge with their bank: 248 disputes, at a \$12 refund plus a \$25 fee each, about \$9,160 a month.
- 40% contact support, at \$5 a contact: about \$3,300 a month.
- So about \$12,460 a month of the \$19,800 is eaten by disputes and support, leaving about \$7,340 a month.
- It also ignores what cannot be priced in this model: lost trust and negative reviews, the chance that many of them would have cancelled a month later anyway, and the legal exposure that some regulators attach to hard-to-cancel flows (section 10).

The ethical nudge test (FW-C10-06). *Transparent:* no, the exit is hidden behind steps. *Reversible:* it is easy to join and hard to leave. *In the user's interest:* no, they would not endorse it. *Symmetric:* no, signing up took one click and leaving takes six. This is a dark pattern ("roach motel," C10.10).

The alternative that serves the business. A short flow with one honest retention step: "Pause for a month or cancel now," a reminder of what they would lose, and a one-click confirmation. Track net outcomes: revenue after refunds and disputes, retention of the people who chose to stay, and complaints and ratings, with the cancellation rate as a diagnostic and not a target ([goodharts-law], C05). Retention of people who actively choose to stay is worth more than retention of people who are stuck.

What this shows. A pattern that raises one metric can lower the outcome that matters, and an honest accounting of its costs is the strongest argument against it.

6. Baseline, Strong, Standout

Prompt: "Review the sign-up form of a banking app. It has 12 fields, and 60% of people who start it do not finish."

Baseline answer

"It has too many fields, so I'd shorten it and make it look cleaner. I'd also use a nicer color for the button."

What is wrong: the problem is stated as taste. No user, no principle, no priority, no check on what each field is for, no test.

Strong answer

"I'd start with the user and the task: a person opening a bank account on a phone who wants to finish in minutes and is wary about giving personal information. I'd walk the form and tie problems to principles: twelve fields at once is a cognitive-load problem (minimal design); fields asked in no clear order and without reasons break 'match the real world'; errors shown only after submit violate 'help with errors' and 'error prevention.'"

"I'd ask what each field is for, and cut any that we do not need at sign-up and can ask later. I'd show progress (visibility of status), validate fields as the user types, use the right keyboard for each input, and keep what they typed when an error occurs. I'd check the states: what happens if the connection drops mid-form, and what an error says. For accessibility: labels for screen readers, contrast and target size."

"I'd test the revised form with five users and measure completion at the form step, with a guardrail on verification failures, since fewer fields may mean less data to verify identity."

Standout answer

Everything in the Strong answer, plus:

"Some fields exist because of regulation (identity checks), so I'd separate those from fields that exist because someone asked for them, and I'd work with the compliance team on what is required at which point. A common design is to split the form: collect the minimum to open the account, and collect the rest progressively once the user has reached something of value. That trades a longer total journey for a higher start-to-finish rate, so I'd measure end-to-end: verified accounts per 100 people who start.

"Trust matters more than usual for a bank: explain why each sensitive field is needed, and show security cues at the moment of asking. For inclusion, think about names (not everyone has a first and last name), addresses (not everyone has a standard format), and people applying with a shared device or on a slow connection.

"I'd reject a trick that lifts completion by hiding costs or pre-ticking consent boxes: it would raise the number and hurt the user. And I'd define success as verified, retained accounts, not completed forms.

"What would change my plan: if the test showed that drop-off was concentrated at the identity-document upload step, then fields are not the issue and the upload (lighting, camera guidance, retry) is."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Ties problems to principles and the user's task	Separates required from habitual fields and works with compliance
Checks states and accessibility	Measures end to end (verified accounts per start) and designs for trust and inclusion
Proposes a test and a guardrail	Rejects manipulative shortcuts and says what would change the plan

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10] .

Company lens: Apple [as of 2026-10]. Apple publishes Human Interface Guidelines for its platforms and a stated emphasis on accessibility features Public [V: Apple Human Interface Guidelines; accessibility features not rechecked] . Third-party guides report that design and critique prompts at Apple reward attention to detail and consistency with the platform Inferred . Practical consequence: refer to platform conventions by principle (consistency, feedback), notice small states and edge cases, and treat accessibility as part of the design.

Company lens: Google [as of 2026-10]. Google publishes the Material Design system and accessibility guidance Public [V: Material Design listing; accessibility guidance not opened]. Candidates commonly report prompts about products used across many devices, languages and connection speeds Inferred. Practical consequence: consider low-end devices and slow connections, and say how the design works across surfaces.

Company lens: Microsoft [as of 2026-10]. Microsoft publishes inclusive-design principles and a toolkit that frames disability as a mismatch between a person and an interface, including permanent, temporary and situational cases Public [V: Microsoft Inclusive Design toolkit manual (Microsoft Design)]. Practical consequence: use the three kinds of limitation to widen who you design for.

Company lens: Amazon [as of 2026-10]. Amazon publishes Customer Obsession and Earn Trust among its leadership principles Public [V: Amazon's Leadership Principles, as cited by several summaries]. Candidates commonly report questions on how you would handle a growth tactic that conflicts with the customer's interest Inferred. Practical consequence: be ready to describe a pattern that raises a metric at the customer's expense, name the cost, and offer an honest alternative (section 10 gives a public case).

Company lens: Meta [as of 2026-10]. Candidates commonly report design prompts paired with questions on engagement, wellbeing and manipulation Inferred [unverified: candidate-reported]. Practical consequence: apply the ethical nudge test to any engagement mechanic, and say what you would measure beyond time spent.

Company lens: Netflix [as of 2026-10]. Netflix publishes technical posts on personalization and interface experimentation Public [V: Netflix Tech Blog posts on experimentation and resilience testing]. Candidates commonly report prompts on choice overload and discovery Inferred. Practical consequence: use Hick's law (more choices slow decisions) and say how you would test a simpler layout.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as emphasizing the quality of its documentation and developer experience, and publishes API documentation and design principles for checkout Inferred [V: Stripe documentation reputation: Slab interview with its former Documentation Manager and Mintlify article; design principles for checkout not separately checked]. Practical consequence: for developer-facing products, treat documentation, error messages and the first integration as the interface.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	"It feels clunky"	No principle, no cost	Name the heuristic and what it costs the user
2	Critiquing taste	Color and style opinions	Tie each problem to a task and a severity
3	Designing only the ideal state	No empty, error or offline behavior	Walk the state map
4	Dictating the solution as PM	You specify layout, not the problem	Give the problem, evidence and criteria
5	Ignoring accessibility	No mention of contrast, labels, targets	Run the pass; name two barriers and fixes
6	Treating a usability test as a statistic	"60% of users fail" from five people	Use it to find problems; size them with analytics
7	Polishing everything equally	No stated quality bar	Use the dial: spend where failure costs most
8	Optimizing a number with a pushier interface	Conversion rises, complaints rise	Run the ethical nudge test; track net outcomes
9	Pre-checked consent or hidden costs	Opt-in by default, fees at the last step	Remove; use honest defaults
10	Leaving help and error text to the end	"Error 4021"	Write plain messages that say what to do

9. Mental models applied

Four models, each tied to a decision in this chapter. The defaults and curb-cut ideas are from C07 and apply here.

Hick's law (MM-hicks-law). The time to decide grows as the number of choices grows. A page with 22 toggles or a menu with 30 items slows people, and grouping, defaults and progressive disclosure reduce the effective number of choices at each moment. It is a reason to ask "what can I remove or defer?" before "what can I add?"

Fitts's law (MM-fitts-law). The time to hit a target depends on its distance and its size: small, far targets are slow and error-prone. It is why touch targets should be large, why the most frequent action belongs within easy reach, and why a destructive action should not sit next to a common one.

The peak-end rule (MM-peak-end). People judge an experience largely by its most intense moment and its end, not by the average. A smooth flow with a frustrating last step (a failed payment, an unclear confirmation) is remembered as a bad one. Spend craft on the end of a task and on the moments of highest emotion (an error, a success).

Defaults and the curb cut (MM-defaults, MM-curb-cut, C07). The default is the product for most people, so choose honest defaults, and a feature designed for a constraint (captions, large targets) tends to help everyone.

Engineering, design and data-science perspectives

- **Engineering.** Ask early what states the system can produce (partial data, timeouts, permission denials), so design can cover them. Performance is UX: a slow screen is a poor one. Raise the cost of custom components against platform standard ones, and agree on the accessibility requirements as acceptance criteria.
- **Design and research.** Designers hold the craft and the pattern knowledge. Ask for options with reasoning, attend the usability sessions and critique against the problem. Agree on a design system and a process for exceptions.
- **Data science.** UX can be measured: task success, time on task, error rate, abandonment at a step, and satisfaction (the HEART framework, C05). Pair these with qualitative tests: numbers say where, observation says why. Define success as a net outcome, not a number a pushy design can lift.
- **Legal, policy and privacy.** Accessibility requirements and rules about subscriptions, consent and deceptive design vary by country and are changing [V: examples: GDPR consent rules, India's CCPA dark-pattern guidelines, EU Digital Services Act]. Involve these colleagues early on sign-up, cancellation, consent and pricing flows.

Checklist: before you approve a design

1. Is the user's task clear, and does the design serve it?
2. Is each problem tied to a principle and a cost, with a severity?
3. Are the empty, loading, error, offline and permission states designed?
4. Did we run the accessibility pass (contrast, labels, targets, not color alone)?
5. Has a design like this been tested with five people from the target segment?
6. Did we choose the quality bar by risk, and record accepted rough edges?
7. Does the ethical nudge test pass: transparent, reversible, in the user's interest, symmetric?
8. Does the metric measure the net outcome and not only the number the design lifts?

10. Case studies

CS-C10-01 · The twelve-field sign-up form · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A budgeting app's sign-up form has 12 fields. Each week 20,000 people start it and 8,000 finish, a completion of 40%. The growth team wants to "make it prettier."

Constraints. Two weeks of design and engineering, a requirement to verify identity for a linked bank account, and a legal need to collect consent at sign-up.

Data. Screen recordings of 15 sign-ups (invented) show that 9 people stop at the phone-number field, asking aloud why it is needed, and 4 stop at an error after submitting that lists all problems at once. Analytics shows that 30% of starters leave at the field for date of birth and address, which is not needed until a bank is linked. A five-user usability test on the current form shows three people unable to understand what to enter in the "Reference" field.

Problem definition. As an observable problem: "People abandon a 12-field form because they do not see why they must give personal details now, and they meet errors only after submitting."

Analysis. The heuristic review (FW-C10-01) ties the problems to principles: too many fields at once (minimal design, cognitive load); no reason given for the phone number (match with the real world and trust); errors after submit and listed in a block (error prevention and help with errors); an unlabeled "Reference" field (match with the real world). Seven of the 12 fields are not needed to create an account; they can be collected when a bank is linked. The identity-verification and consent fields are required and stay. Cosmetic changes address none of these.

Options.

Option	Optimizes	Costs
A. Restyle the form	Appearance	No change in the cause
B. Cut to six fields now, add reasons for sensitive ones, validate as the user types, keep progress on error	Completion	Later steps collect the rest; two weeks of work
C. B plus a progress indicator and a save-and-resume link	Completion and recovery	More engineering

Tradeoffs. B addresses the observed causes; moving fields later shifts some drop-off to later steps, so end-to-end is what to measure. C adds value at more cost; test B first.

Decision. Build B. Primary metric: verified accounts per 100 people who start. Guardrail: the share of completed forms that fail verification (baseline 4%). Counter metric: abandonment at the later step where the removed fields are now asked.

Outcome. In this fictional scenario completion rose from 40% to 58%, so 11,600 of 20,000 finish; verification failures rose from 4% to 6% because there was less data up front. Verified accounts: before $8,000 \times 0.96 = 7,680$; after $11,600 \times 0.94 = 10,904$, up 3,224 or 42%. Drop-off at the later step rose modestly, and the net remained clearly positive.

Alternative outcomes. If verification failures had doubled to 12%, verified accounts would have been $11,600 \times 0.88 = 10,208$, still above the old 7,680, and the team would have looked at which removed field helped verification. If completion had not moved, the usability test would have been repeated on the new form to find the problem the review missed. The lesson: find the specific causes by watching and by principle, and measure end to end.

CS-C10-02 · Amazon Prime and the FTC, 2023 to 2025: enrollment, cancellation and dark patterns · Verified · Advanced

Sources (types): the US Federal Trade Commission's press release announcing the settlement (25 September 2025) and its refunds page for the settlement (primary, for what the FTC alleged and what the order requires); a law-firm summary and CNBC's report of the amended complaint filed on 20 September 2023 (secondary; the complaint itself was not read); and reports of Amazon's statement on the settlement (The Register and HomePage News). This case describes allegations and a settlement; it does not describe a court's finding of liability. The analysis is the author's. Re-read in October 2026.

Situation. In June 2023 the FTC sued Amazon, and amended its complaint in September 2023, alleging that Amazon used "dark patterns," meaning manipulative or deceptive user-interface designs, to enroll consumers in its automatically renewing Prime subscription, and made cancellation hard through a multi-step flow that Amazon internally called "Iliad," after Homer's epic about a long war. A law-firm summary of the amended complaint, filed on 20 September 2023, says it alleged that about 35 million consumers were unknowingly enrolled [V: law-firm summary, 2025]; in announcing the settlement, the FTC described redress for an estimated 35 million consumers impacted by unwanted Prime enrollment or deferred cancellation [V: FTC press release, 25 September 2025]. On 25 September 2025, days after a trial in Seattle began (as a payments trade outlet reported), Amazon agreed to a stipulated final order of \$2.5 billion, which also covers two Amazon executives named by the FTC: a \$1 billion civil penalty and \$1.5 billion in refunds to affected consumers, together with requirements to end the alleged enrollment and cancellation practices, to give customers a clear and conspicuous button to decline Prime, to disclose the cost, the date and frequency of charges, auto-renewal and cancellation procedures during enrollment, to offer a cancellation method as easy as the one used to sign up, and to pay for an independent supervisor of the refund process [V: FTC press release]. Amazon said that it and its executives "have always followed the law," and reports say it did not admit wrongdoing [V: Amazon statement as reported, September 2025].

Constraints. A subscription's value to the company depends on how many people enroll and stay; interface choices affect both. The law the FTC invoked includes the Restore Online Shoppers' Confidence Act, which governs online subscriptions, and the FTC Act's ban on deceptive practices [V: FTC press release]. The facts at issue came partly from internal company documents produced in litigation, as reported, which the sources used here describe only in summary.

Data. As reported: the complaint's allegations about enrollment designs (for example, interfaces that made it hard to decline Prime while checking out) and about the cancellation flow (a journey of several pages and many clicks, which an FTC official described as long and deliberately labyrinthine); the settlement terms above; a refund eligibility window of 23 June 2019 to 23 June 2025 for people who signed up through the challenged enrollment flows or unsuccessfully tried to cancel, with automatic refunds by 25 December 2025 followed by a claims process [V: FTC refunds page]; later reports quoting the FTC and the settlement site say automatic payments went out from 12 November to 24 December 2025 and that the

claims process opened on 5 January 2026 [V: Newsweek and a settlement-site summary, checked October 2026]; the FTC's statement that the \$1 billion penalty is the largest ever in a case involving an FTC rule violation [V: FTC press release].

Problem definition. *The author's reading.* The case concerns a design question that PMs face: is an interface that raises enrollment and retention acceptable if users would not endorse it? As an observable problem: "Joining took one click; leaving took many steps, and some people said they never meant to join."

Analysis. *The author's analysis.* Read through this chapter's ethical nudge test (FW-C10-06), the alleged patterns fail all four questions: not transparent (enrollment was allegedly confusing), not reversible in practice (hard to cancel), not in the user's interest (users would not have endorsed it), and not symmetric (one click in, many out). They match two entries in the dark-pattern table: hard to cancel, and misdirection. The pull toward such designs is a metrics story: subscriber counts and retention are easy to measure and reward, and a flow that raises them can look like success until its costs arrive ([goodharts-law], C05). Worked example 4 shows the arithmetic at small scale: revenue from people who were stuck is partly eaten by disputes and support, before counting trust and legal exposure. In this case the exposure was a settlement of \$2.5 billion, far larger than the revenue any single design change could have produced, which shows how the net outcome can be the opposite of the number the pattern lifted.

The case also warns against overreading. These are allegations, resolved by settlement without a trial verdict, and Amazon's position, as reported, is that it did nothing wrong. A fair use for a PM is as a public record of what a regulator considers unacceptable and of how large the consequences can be, not as proof of anyone's intentions.

Options. *The author's reconstruction of choices a team could have faced; not a record of Amazon's deliberations.*

Option	Optimizes	Costs
A. Keep flows tuned for enrollment and retention	Subscriber counts in the short term	Complaints, disputes, trust, and regulatory and legal exposure
B. Disclose terms clearly at sign-up and make cancellation as easy as enrollment	Trust and compliance	Some lost subscribers in the short term
C. Keep an honest retention step (pause, a clear statement of benefits) with a one-click exit	Retention of people who choose to stay	Design effort; fewer stuck subscribers

Tradeoffs. A keeps the number up and carries the costs listed. B and C lower the cancellation barrier and give up the subscribers who were stuck, in exchange for a base that stays by choice and a much smaller legal risk.

Decision. As documented, the parties settled on the terms above, which require changes to enrollment disclosures and cancellation [V: FTC press release].

Outcome. Reported: the \$2.5 billion settlement, refunds to eligible consumers and required changes to the flows. How much the changes affect subscriptions or revenue is not established in the sources used here.

Alternative outcomes. *Inference.* Had the design been simple from the start, the short-term subscriber count might have been lower and the costs of disputes, refunds and enforcement avoided. Had the case gone to trial, a court might have found for either side; the settlement means no judgment tests the allegations. The method to carry away: apply the ethical nudge test to any flow that touches enrollment, consent or cancellation, count net outcomes, and involve legal and policy colleagues early.

11. Interview questions and model answers

Seven questions: two Beginner, two Intermediate, three Hard. Model answers are illustrative; every figure is an assumption, and the arithmetic was computed. None is tied to a company: UX reasoning is asked at all of them, and the lenses in section 7 describe the emphasis. The answers are fuller than a first pass in a real round.

IQ-C10-01 · Beginner · product design · PS

Prompt. We have a designer. What is the PM's role in UX?

Rubric.

- **Baseline:** "The PM decides what to build and the designer makes it look nice."
- **Strong:** Splits ownership of the problem and the solution, and names what the PM contributes.
- **Standout:** Gives examples of problem framing versus solution dictating, mentions research, tradeoffs and decisions, and says how disagreements are settled.

Model answer. I'd say the PM owns the problem and the outcome, and the designer owns the craft of the solution. My job is to make the problem clear: who the user is, what they are trying to do, what success looks like, what constraints we have (time, platform, technology, policy), and what we have learned from research and data. The designer's job is to turn that into a flow and an interface that works and feels right. The engineer tells us what can be built well.

The difference shows in how I give input. "Make the button blue and move it to the top" hands the designer a solution and removes their craft. "Forty percent of first-time users stall at the password step; I want 70% to finish, and we cannot add a screen" gives them a problem and criteria, and they may find a better answer than I would have, such as showing the rules as people type.

My other contributions: attending research sessions so that I see users struggle first-hand, making the tradeoffs and calls on scope, checking designs against the problem and not against taste, and bringing engineers in early. When we disagree, I'd propose a quick test

with five users and let evidence decide before rank does. I'd keep the decision on what to build and the success criteria, and I would defer to the designer on how it should look and work unless it fails the problem.

Follow-ups.

- **Q.** The designer and you disagree about a solution. How is it settled? **A.** "By going back to the user problem and the evidence, not by rank. I would restate the problem and the success measure we agreed, ask what each of us expects users to do and why, and propose the cheapest test that distinguishes the two ideas, such as a prototype with five users or a quick A/B test. If the test cannot be run, the decision rights apply: the designer decides how the interaction works, I decide whether and when it ships against priorities, and I say so openly. I would also check whether my preference is a solution I brought without a problem statement."
- **Q.** Give an example of framing the problem versus dictating the solution. **A.** "Dictating: "Add a progress bar to the sign-up form." Framing: "Thirty-eight percent of people leave the sign-up form at step three, and interviews suggest they do not know how much is left or why we need the information; we want completion above 55% without more support contacts." The second leaves the designer free to try a progress bar, a shorter form or a better explanation, and gives a measure to judge it. It also gives engineering the constraint (two weeks) so that the design fits."

IQ-C10-02 · Beginner · product design · PS

Prompt. A checkout screen shows "Error 4021" when a payment fails. What is wrong, and what do you do?

Rubric.

- **Baseline:** "The message is ugly; make it friendlier."
- **Strong:** Names the heuristic (help with errors), the cost to the user, and a better message.
- **Standout:** Looks at the whole state (recovery, preserved data, support, tracking), prioritizes by impact and says how to measure the fix.

Model answer. The principle is help with errors: an error should say, in plain words, what went wrong and what the person can do. "Error 4021" tells the user nothing, so I'd expect them to retry blindly, abandon, or contact support. At checkout, the cost is a lost order and, at a payment, a loss of trust: people worry whether they were charged.

A better message and the state around it. "Your card was declined. Check the number and expiry, try another card, or use a different payment method. You have not been charged." Plus: keep everything they typed and the cart intact, put the cursor at the field most likely wrong, offer alternative payment methods as buttons, and show a way to get help. The code can stay as a small reference for support.

Different failures need different messages. A declined card, a network failure and a provider outage have different fixes. A single message for all of them is a smaller version of the same problem, so I'd ask engineering to map the codes to the main cases.

Priority and measurement. This is a high-severity state because it sits at the revenue moment. I'd measure the share of failed payments that lead to a completed purchase within the session (recovery rate) and the support contacts about payment, and test the new message with five users shown a declined card. I'd also check accessibility: the message is announced by screen readers and does not rely on color alone.

Follow-ups.

- **Q.** How would you prioritize the checkout error fixes? **A.** "By impact. Count the error types by frequency and by lost orders: if card declines are 70% of payment failures and the cost of a lost order is \$20, a clear message there is worth much more than polishing a rare error. I would fix first the most frequent errors with the largest cost, write messages that say what happened and what to do, keep the cart and entered data after an error, and offer another payment method in one tap. Lower-frequency errors can follow, grouped by cause. The test is the effect on completed payments after a failure."
- **Q.** How would you measure that the fix worked? **A.** "With recovery rate: of users who see a payment error, the share who complete an order within the same session or 24 hours. Compare before and after, or A/B test the new message on a random half. Secondary measures are support contacts about payments and double-charge worries (for example, contacts asking whether they were charged), and the guardrail is that the number of payment attempts does not rise in a way that suggests blind retrying. A recovery rate rising from 35% to 45% would be a strong result for a message change."

IQ-C10-03 · Intermediate · product design · PS

Prompt. The deadline is in two weeks and the UX is rough. Do you ship?

Rubric.

- **Baseline:** "Ship, we can fix it later," or "No, quality comes first."
- **Strong:** Sets a quality bar by layer and by risk, and decides what to cut.
- **Standout:** Separates accessibility, safety and honest information (not negotiable) from polish, records accepted rough edges with a date, and sets a measure and a rollback.

Model answer. I'd decide by risk, using the quality dial (FW-C10-05): works, usable, delightful. I'd ask three things: what is the cost if a user fails here, how many users are affected, and what do we learn by shipping now?

Not negotiable. The feature must work (correct, reliable), the core path must be usable, and accessibility basics (labels, contrast, targets) and honest information (clear pricing, clear errors) must be in place. These are not polish. States matter too: the empty and error states are where users get stuck, so they get at least a plain message and a next step.

Negotiable. Animations and micro-interactions, secondary screens, visual refinement, edge-case flows that few people reach. I'd cut those first.

Make it explicit. I'd write a short list of accepted rough edges with an owner and a date to fix them, since polish debt grows if no one tracks it. I'd release in stages (a small share of users first) with a guardrail on task completion and support contacts and a rollback ready, so a rough UX reaches few people before we learn how rough.

Decision. "Ship the core path with accessibility and states intact in two weeks to 10% of users; defer the animations and the settings polish to the next sprint, with a date." What would change my mind: if the rough area is a trust moment (payment, privacy) or the first-run experience, I'd hold the date, because failures there cost more than a late launch.

Follow-ups.

- **Q.** Which rough edges do you accept for the launch, and how do you keep them from being forgotten? **A.** "I accept visual polish and minor friction on low-traffic paths: spacing, animations, secondary copy. I do not accept failures in the core task, accessibility basics or misleading information. For each accepted rough edge I record it in a short list with the user impact, an owner and a date to fix, usually within one or two sprints, and review the list at the first post-launch check. Rough edges that are never recorded become permanent. I would also tell stakeholders what is rough, so that surprises are not discovered by users first."
- **Q.** What measure and rollback would you set? **A.** "A primary measure for the core path, such as task completion rate, with guardrails on error rate, support contacts and accessibility defects found in a quick audit. Release to a small percentage first, say 5%, then widen after a few days of stable numbers. The rollback trigger is stated in advance: for example, completion below 90% of the control group's, or error rate above twice baseline. If the trigger fires, roll back, fix and re-release; the discipline is that the rule is written before launch and does not change on launch day."

IQ-C10-04 · Intermediate · product design · PS

Prompt. Your designer wants a custom date picker for a signature feel. Your engineer wants the platform's standard one. What do you decide?

Rubric.

- **Baseline:** "Go with the designer," or "pick the cheaper one."
- **Strong:** Weighs user value, consistency, accessibility, cost and maintenance, and tests.
- **Standout:** Sees that standard components carry accessibility and familiarity, proposes a compromise (custom only where it matters), sets a test and a decision rule, and keeps the relationship in view.

Model answer. I'd not decide by rank. I'd frame the question around the user and the cost. What task does the date picker serve, and is it a differentiator (for example, a travel app where date selection is the core action) or a supporting control (a birthday in settings)?

For standard: people already know it (consistency and standards), it comes with accessibility support (screen reader labels, keyboard and voice operation), it is cheaper to build and maintain, and it updates with the platform. *For custom:* it can make a core task faster or clearer, and it expresses the brand. Its costs: building and testing for accessibility, more bugs, and upkeep across devices.

How I'd settle it. A quick test: prototype both and watch five users pick a date range for the core task. Measure time to complete, errors and preference. If the custom picker is clearly better at the core task, and the team commits to accessibility and maintenance, it is justified. If not, the standard wins.

A compromise. Use standard components for supporting controls and invest in a custom one only where the task is central. I'd record the decision and the reason so it is not reopened without new evidence. I'd tell the designer and engineer that both views are right about something: one is about the user's task and the brand, the other about cost, accessibility and maintenance. What would change my mind: a usability result that contradicts my expectation.

Follow-ups.

- **Q.** What decision rule would you use between the custom and standard date picker? **A.** "Whether the control is the core of the experience. If dates are the central action, as in a travel search, a custom picker may justify its cost, if it passes a test: task time no worse than the standard, error rate no higher, and no accessibility regressions on screen reader and keyboard use. If dates are a supporting control, use the standard one. I would run a quick prototype test with eight users on both, plus an accessibility check, and decide on the numbers. The rule is written before the test so the loudest opinion does not win."
- **Q.** How do you keep the relationship with the designer and engineer intact? **A.** "By treating it as a shared problem and not a contest. I would thank the designer for the aim (a distinctive feel), ask the engineer for the cost of each option and the maintenance burden, and propose the compromise: standard component by default, custom styling where the standard allows, custom behavior only where tested. I would make sure the decision and its reason are written down and that the designer sees what would change it. If the standard is chosen, the designer gets another place to express the feel, so that the outcome does not read as a loss."

IQ-C10-05 · Hard · product design · PS

Prompt. The growth team wants a pre-checked box that opts users into marketing emails. They expect opt-in to more than double. What do you do?

Rubric.

- **Baseline:** "Do it, since it doubles opt-in," or "refuse" with no reasoning.
- **Strong:** Applies an ethical test, mentions the legal and trust risk, and proposes an alternative.
- **Standout:** Quantifies net outcomes (engaged recipients, complaints), proposes a better-timed ask, sets guardrails and says how to take the conversation to the growth team.

Model answer. I'd start by taking the growth team's goal seriously: they want more people who receive and value marketing messages. Then I'd run the ethical nudge test (FW-C10-06). A pre-checked box is not transparent (many people do not notice it), is easy to leave only if you spot it, is not something users would endorse if they understood, and is not symmetric. It is a dark pattern. In several jurisdictions, consent rules require an affirmative action to opt in, so it may also be unlawful [V: GDPR Article 4(11) (clear affirmative action) and the ePrivacy Directive (opt-in for marketing email), as summarized in guidance], and I'd ask legal and privacy teams immediately.

Count net outcomes. Suppose an unchecked box gets 18% opt-in and a pre-checked box 41% (invented). The pre-checked list is bigger, but its members did not choose it: say 9% of them open the emails, against 31% of the people who chose. Engaged recipients per 100 sign-ups: $18 \times 0.31 = 5.6$ versus $41 \times 0.09 = 3.7$. The honest approach produces more engaged recipients. The pre-checked list also generates more unsubscribes and spam complaints, which damage the reputation of our sending domain and lower delivery to everyone.

An alternative that serves the goal. Ask at a better moment, with a clear value: after the first order, "Want order updates and occasional offers? Yes / No thanks," unchecked, with an easy way to change it later. Test the wording and timing. Measure engaged recipients per 100 sign-ups, complaints and unsubscribes, with opt-in rate as a diagnostic and not the target ([goodharts-law]), C05).

How I'd say it. "Opt-in is the wrong number. The right number is engaged subscribers, and the checkbox lowers it while raising legal risk. Let's test a better ask." What would change my mind: if the test showed that the honest ask gets too few opt-ins to matter, I'd look at the value we are offering, not at hiding the choice.

Follow-ups.

- **Q.** What would you propose instead of the pre-checked box? **A.** "An unchecked box with a clear value statement, shown at a moment when people understand why they might want it, such as after the first successful purchase: Get order updates and offers? with one tap to say yes and one to decline. I would test it against the pre-checked version and compare engaged recipients (opens and clicks in 60 days), unsubscribes and spam complaints, not just opt-in rate. A pre-checked box may double opt-ins, but if half of them never open and 3% mark messages as spam, deliverability falls and the list is worth less."
- **Q.** How do you take this conversation to the growth team? **A.** "As a shared goal. I would agree that we want more recipients who value the messages, show the numbers from a test (opt-ins, engaged recipients, complaints, deliverability), and explain the legal and trust risk in plain terms: consent rules in several regions require an affirmative action, and complaints can reduce the reach of all our emails. I would offer an alternative that raises opt-in through better timing and value, and keep guardrails on complaints. If leadership still wants the pre-checked box, I would escalate the legal question to counsel, since it is a risk beyond my authority to accept."

IQ-C10-06 · Hard · product design · PS

Prompt. Leadership wants to skip accessibility work to make the launch date. What do you say?

Rubric.

- **Baseline:** "Accessibility is the law" or "okay, we'll do it later."
- **Strong:** Explains the users affected, the cost of retrofitting, and a minimum bar.
- **Standout:** Quantifies the audience, separates launch-blocking items from later improvements, notes legal exposure and brand risk, offers a dated plan and a scope trade.

Model answer. I'd treat it as a scoping conversation with data. First, who is affected: people who use screen readers, voice control, larger text, captions or switch devices, and, by the inclusive-design framing, people with temporary and situational limits (a broken arm, bright sun, a noisy room). Suppose we estimate that 8% of our users use assistive features or larger text (invented), which is 80,000 of a million. That is not an edge case.

Cost. Retrofitting accessibility after launch usually costs far more than building it in, because it touches structure (labels, focus order, contrast, targets) that is expensive to change once shipped. Skipping it also creates risk: legal exposure in some markets [V: consent and children's data rules differ by country (GDPR, COPPA, India's DPDP Act)], app-store and partner requirements, and brand damage when people who cannot use the product say so.

A minimum bar that does not threaten the date. Launch-blocking items: labels for every control and image for screen readers, contrast of 4.5:1 for normal text (a designer can check in minutes), touch targets of adequate size, keyboard and voice operation of the core path, and no information carried by color alone. These are small if built in from the start. Defer to a dated plan: enhancements to advanced features, deeper testing with assistive-technology users, and nonessential motion refinements.

What I'd offer. "We can hold the date if we protect the core-path basics, which add about three days, and defer the advanced items to a dated plan with an owner. If we skip the basics we will spend more later and risk exclusion on day one." I'd bring a trade: cut a non-core feature before cutting accessibility. What would change my mind: if the launch is a small, internal pilot with no public users, I'd allow a later bar for the pilot, and I'd write down the date by which it must meet the standard.

Follow-ups.

- **Q.** How do you separate launch-blocking accessibility items from later improvements? **A.** "By impact on task completion. Blockers are things that prevent a user with assistive technology from completing a core task: missing labels on controls, keyboard traps, insufficient contrast on key text, targets too small to hit, and content that cannot be read by a screen reader. Later items are enhancements: refined focus order on rarely used screens, improved captions styling, extra shortcuts. I would run a quick audit with automated tools plus a manual pass by someone who uses a screen reader, list defects by severity, and agree with engineering which are blockers. The list and its dates go to leadership."
- **Q.** What do you offer leadership instead of skipping the work? **A.** "A dated plan with a scope trade. Ship on time with the blockers fixed and a smaller feature set, or keep the full scope and move the date by one sprint. I would show the numbers: if 8% of users use

assistive features or larger text, that is 80,000 of a million, retrofitting after launch costs more than building in, and legal and brand exposure are real. Then ask for the decision. A choice presented as two options with costs is easier to accept than a refusal, and I would put the accepted risks in writing."

IQ-C10-07 · Hard · product design · PS

Prompt. In a usability test, two of five people failed a critical task. The team says five is too small a sample to act on. How do you respond?

Rubric.

- **Baseline:** "Five is enough," or "we need more users."
- **Strong:** Explains what the test is for (finding problems) and what it cannot do (sizing them).
- **Standout:** Gives the discovery argument and the interval, separates the existence of a problem from its rate, proposes analytics and a cheap retest, and decides on cost.

Model answer. The team is right about one thing and wrong about another. A five-person test cannot tell us how common the failure is: two of five is 40%, but the 95% interval for that is roughly 12% to 77%, so the true rate could be small or large. We should not report "40% of users fail."

But the purpose of the test is to find problems, not to count them. If each participant finds about 31% of the problems on average (the commonly reported figure [V: Nielsen and Landauer (1993): one user finds about 31 percent of problems]), five participants find about 84% of them ($1 - 0.69^5$). Seeing two people fail a critical task, and seeing why, is strong evidence that a problem exists and what causes it. The question is not whether it exists; it does. The question is how much it matters and how cheaply we can fix it.

What I'd do. First, look at the cause: if both failures came from the same visible issue (a hidden control, an unclear label), the fix is probably cheap, and I'd fix it without waiting. Second, size it with analytics: what share of live users reach that step and abandon, and how does it compare with other steps? Third, run another round with five new participants after the fix, since people who failed now know the answer.

Decision. If the fix costs a day, make it now. If it costs a month, size the problem first with analytics and weigh it against other work. I'd tell the team: "Five people cannot give us a rate, and they have shown us a real problem and its cause. Let's fix the cheap part and measure the rest." What would change my mind: if the two failures had different causes, or if analytics showed almost no one reaches that step.

Follow-ups.

- **Q.** How do you separate whether a problem exists from how common it is? **A.** "Two failures out of five tell me with high confidence that the problem exists and tell me little about its rate. The interval for 2 of 5 is about 12% to 77%. For the rate, I would use analytics: the funnel for that task across all users, which shows the share who drop at that step, and a larger unmoderated test. For the existence question, a reproducible

failure with a clear cause (for example, a button that looks disabled) is evidence enough to fix, and I would say so in the report while stating the rate as unknown."

- **Q.** How would you decide whether to fix it? **A.** "On cost and impact. Estimate the share of users on the critical task (from analytics), the failure rate range, and the cost of the fix. If the task is critical, the plausible failure rate is meaningful, and the fix takes two days, fix it and retest with five more users: a fix that costs less than the uncertainty is not worth debating. If the fix is large, run the analytics check first to see the rate. The decision rule: fix now if the fix is under a week and the task is critical; otherwise measure first, with a date."

12. Exercises

Six exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C10-01 · Review a device-settings screen

Problem. A device's settings screen (described, invented) has 40 switches in one scrolling list, a switch labeled "Allow" with no object, a "Restore defaults" button next to "Save" that acts without confirmation, gray-on-white labels at a contrast of 3.0 to 1, and a custom control for brightness that works differently from the device's other sliders. Review it against the ten heuristics and the platform's published guidelines as a reference, rank by severity and propose a fix order.

Context. The setting is a device-maker's settings screen; no claim is made about any real product. Platform guidelines are a public reference [V: Apple Human Interface Guidelines and Material Design].

Data. As above.

Constraints. Eight minutes.

Expected approach. The heuristic review (FW-C10-01) and the accessibility pass (FW-C10-03).

Solution. (1) "Restore defaults" next to "Save" with no confirmation or undo: user control and error prevention; blocks (data loss); fix first: separate it, confirm, or offer undo. (2) 40 switches in one list: recognition over recall, minimal design, cognitive load; slows; group by purpose and add search. (3) "Allow" with no object: match with the real world, visibility; slows; label "Allow notifications from this app." (4) Labels at 3.0:1: accessibility (contrast needs 4.5:1 for normal text); blocks for some; darken to at least #767676 on white (4.54:1). (5) A custom brightness control that differs from the other sliders: consistency and standards, and the platform's guidelines suggest using the standard slider; annoys; use the standard control unless the custom one tests better. Order: 1, 4, 2, 3, 5.

Alternative solution. Rank the contrast issue first if the audience includes many older users, since it blocks reading.

Tradeoffs. Fixing everything takes time; the order by severity puts irreversible harm and exclusion first.

EX-C10-02 · A state map for photo upload

Problem. A photo-upload feature lets a user pick a photo, upload it to a profile and see it appear. List all eight states (FW-C10-02) and what the user should see in each.

Context. Mobile app, variable connection.

Data. None.

Constraints. Six minutes.

Expected approach. The state map.

Solution. Ideal: the photo appears with a confirmation. Empty: no photo yet; show a placeholder and "Add a photo." Loading: a progress indicator with percentage and a way to cancel. Partial: if the thumbnail fails, show the text and a placeholder; upload continues. Error: "Upload failed. Check your connection. Your photo is saved; try again" with a retry button. Offline or slow: queue the upload and say "Will upload when you're back online." Permission denied: if photo access was denied, explain why it is needed and link to settings. Success: "Photo updated" with Undo.

Alternative solution. Add a state for "too large a file" with a message and an automatic resize.

Tradeoffs. Every state costs design and build time; the error, offline and permission states are the ones that most often cause abandonment.

EX-C10-03 · Contrast ratios

Problem. Using the usual minimums (4.5:1 for normal text, 3:1 for large text), classify these text colors on a white background, given measured ratios: (a) #767676 at 4.54; (b) #777777 at 4.48; (c) #0066CC at 5.57; (d) #808080 at 3.95; (e) #959595 at 3.0. For each, say whether it passes for normal text and for large text.

Context. A designer asks for quick feedback.

Data. As above.

Constraints. Three minutes.

Expected approach. The accessibility pass (FW-C10-03).

Solution. (a) Passes both ($4.54 \geq 4.5$). (b) Fails normal text by a hair ($4.48 < 4.5$); passes large text. (c) Passes both. (d) Fails normal text ($3.95 < 4.5$); passes large text (≥ 3). (e) Fails normal text; passes large text only at exactly 3.0, with no margin. The practical advice: choose colors with margin (for example, #6B6B6B at 5.33 rather than #767676), because rendering and screens vary.

Alternative solution. Recommend that the design system define a small set of approved text colors that pass, so individual designers do not need to check.

Tradeoffs. Higher contrast is less subtle for the brand; accessible palettes are a design constraint, and the curb-cut effect means they also help in bright light.

EX-C10-04 · Read a five-user test

Problem. Five users try three tasks: (a) find a product, 5 of 5 succeed; (b) apply a coupon, 1 of 5 succeeds; (c) change delivery address, 3 of 5 succeed. Rank the problems, say what you can and cannot conclude about rates, and say what to do next.

Context. A prototype checkout.

Data. As above. The 95% interval for 4 failures out of 5 is roughly 38% to 96%; for 2 failures out of 5, roughly 12% to 77%.

Constraints. Five minutes.

Expected approach. The five-user test (FW-C10-04).

Solution. Rank: (b) coupon, four of five failed, the biggest; (c) address, two of five failed; (a) fine. Conclusions: we can say the coupon task has a severe problem, and the observations should tell why (where people looked, what they tried). We cannot say that "80% of customers fail": the interval is about 38% to 96%. Next: fix the cause, measure with analytics the share who reach and abandon at the coupon step on the live product, and retest with five new users.

Alternative solution. If the coupon problem is cheap to fix and the address problem is expensive, fix the coupon at once and size the address issue first.

Tradeoffs. Waiting to size every problem delays obvious fixes; fixing without checking the cause risks missing it.

EX-C10-05 · Spot the dark patterns

Problem. Name the pattern and propose an honest alternative: (a) a free trial that converts to a paid plan with no reminder; (b) a "No thanks, I prefer paying full price" button; (c) a fee added on the last step of checkout; (d) a cancel link in tiny gray text on the fifth screen; (e) a pop-up that reappears every visit after being dismissed.

Context. A design review.

Data. None.

Constraints. Five minutes.

Expected approach. The ethical nudge test (FW-C10-06) and the dark-pattern table (C10.10).

Solution. (a) Forced continuity: send a reminder before the trial ends and say the price at sign-up. (b) Confirmshaming: neutral wording ("No thanks"). (c) Hidden costs: show the total from the start. (d) Hard to cancel with misdirection: a clear cancel link in the account page and a short flow. (e) Nagging: respect the dismissal for a long period or ask a different question later.

Alternative solution. For (a), offer a "remind me" option at sign-up and a one-click cancel in the reminder.

Tradeoffs. Honest versions cost some short-term conversion and reduce complaints, refunds and risk; measure net outcomes.

EX-C10-06 · Set the quality bar

Problem. For each feature say the quality bar for works, usable and delightful, and what you accept as rough: (a) an internal admin report used by five staff; (b) the payment screen of a consumer app; (c) a first-time-user onboarding tour for a new product.

Context. Limited time in a sprint.

Data. None.

Constraints. Five minutes.

Expected approach. The quality dial (FW-C10-05).

Solution. (a) Works: high. Usable: adequate. Delightful: low; accept a plain layout, no animations. (b) Works: very high. Usable: high; clear errors, honest totals, accessibility. Delightful: moderate, with the confirmation moment (peak-end rule). Accept rough: nothing in the core path. (c) Works: high. Usable: high; short, skippable, with a clear next step. Delightful: moderate to high since first impressions shape retention; accept a less polished second language version if it follows a date.

Alternative solution. For (c), test a lighter onboarding (one tip) against the tour before investing in polish.

Tradeoffs. A lower bar is faster; record the accepted rough edges and a date to revisit them.

13. Practice protocol

Timed drills (three a week for two weeks).

- *Week 1: see and name.* Take three screens from products you use. For each, spend ten minutes on the heuristic review (FW-C10-01) and the state map (FW-C10-02), naming each problem with its principle and cost. Use IQ-C10-02 and EX-C10-01.
- *Week 2: judge and decide.* Use IQ-C10-03, IQ-C10-05, IQ-C10-06, IQ-C10-07 and EX-C10-05. Practice the quality dial, the ethical nudge test and the accessibility minimum bar aloud, with a partner who pushes back with deadlines and metrics.

With a partner. Ask your partner to say "conversion would go up" at some point, and answer with the ethical nudge test and a net outcome. Practice saying "I'd test that with five people" in a sentence.

Observe real users once. Run a five-person usability test of any product, with three tasks. Record where people hesitate. Nothing replaces watching.

Feedback checklist (yes, partly, no).

1. Did I start from the user's task?
2. Did I name each problem with a principle and its cost?
3. Did I check the states beyond the ideal?
4. Did I raise accessibility with specific barriers and fixes?
5. Did I set a quality bar by risk and say what I accept?
6. Did I run the ethical nudge test on any steering design?
7. Did I propose a test and a metric with a guardrail?
8. Did I finish with the two-sentence close from section 4?

Answer journal. For each drill record the date, the product, the top three problems with principles, the state you missed and your score out of ten.

14. Implementation playbook

Apply the chapter on real work over the next two weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Run the heuristic review and the state map on one screen of your product. Share the list with the designer, ranked by severity, and ask what they would add.

Week 1. Run the accessibility pass (FW-C10-03) on one flow: check contrast, labels, target size and keyboard operation. Fix one cheap issue and log the rest.

Week 1. Watch five users, or five recordings, attempt one task in your product. Write down each hesitation. Rank by severity and share with your team.

Week 2. Look for a flow in your product that fails the ethical nudge test (a pre-checked box, a hard-to-find cancel, a nag). Write the net-outcome case for changing it and the honest alternative.

Week 2. Write a quality bar (FW-C10-05) for the next feature: where you will spend and what you will accept as rough, with a date to revisit.

Deliverables that become stories: the ranked heuristic review (a time you found and prioritized usability problems), the five-user observation (a time you watched users and changed a decision), and the net-outcome case against a dark pattern (a time you protected users and the business). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. The PM owns the problem and the outcome, the designer owns the craft; give criteria, not layouts, and settle disagreements with a quick test.
2. Name usability problems with a principle and a cost, and review the states beyond the ideal: empty, loading, error, offline, permission denied.
3. Treat accessibility as part of the main path: a short pass on contrast, labels, targets and non-color cues catches most barriers, and it helps everyone.
4. Set the quality bar by risk: spend on the core task, trust moments and accessibility; accept recorded rough edges elsewhere.
5. Apply the ethical nudge test to any design that steers behavior, and measure net outcomes, because a pattern that lifts one number can hurt the one that matters.

Further reading

- Don Norman, *The Design of Everyday Things* (revised edition, 2013). Affordances, signifiers, feedback and the conceptual model.
- Steve Krug, *Don't Make Me Think* (revised edition, 2014). Short and practical on usability and testing.
- Jakob Nielsen, "10 Usability Heuristics for User Interface Design," Nielsen Norman Group [V: Nielsen Norman Group article; heuristics developed with Rolf Molich (1990), refined by Nielsen in 1994 from 249 usability problems, article updated 2020]. The source for FW-C10-01.
- World Wide Web Consortium, Web Content Accessibility Guidelines (WCAG), current version 2.2 [V: W3C, WCAG 2.2 Recommendation (latest dated version 12 December 2024)]. The public standard behind FW-C10-03.
- Apple, *Human Interface Guidelines*; Google, *Material Design*; Microsoft, *Fluent design* and the Inclusive Design toolkit. Public platform references [V: Apple Human Interface Guidelines and Material Design; Fluent not rechecked].
- Kat Holmes, *Mismatch: How Inclusion Shapes Design* (2018). Inclusive design and the idea of mismatch.
- Richard H. Thaler and Cass R. Sunstein, *Nudge* (2008). Choice architecture and its ethics.
- Harry Brignull and others, *deceptive.design* (formerly *darkpatterns.org*). A catalog of dark patterns [V: Brignull coined dark patterns in 2010; the site is now *deceptive.design*].
- US Federal Trade Commission, press release on the Amazon Prime settlement (25 September 2025) and the FTC's refunds page for the settlement (read); the complaint filed June 2023 and amended 20 September 2023 (known through the FTC's announcements and press summaries, not read directly). Sources for CS-C10-02.